

Experiment No: 07

Course Code: CSE-406

*Course Title: Digital Image Processing Laboratory
4th Year 1st Semester Examination 2023*

Date of Performance: 06-10-2024

Date of Submission: 20-10-2024



Submitted to-

Dr. Morium Akter

Professor

Dr. Md. Golam Moazzem

Professor

*Department of Computer Science and Engineering
Jahangirnagar University
Savar, Dhaka-1342*

Sl	Class Roll	Exam Roll	Name
01	364	202176	Suraiya Mahmuda

Experiment Name:

- a. Dilation of an image.
- b. Erosion Operation of an image.
- c. Opening and Closing Operation of an image.
- d. FFT operation of an image.
- e. Homomorphic Transform of an image.

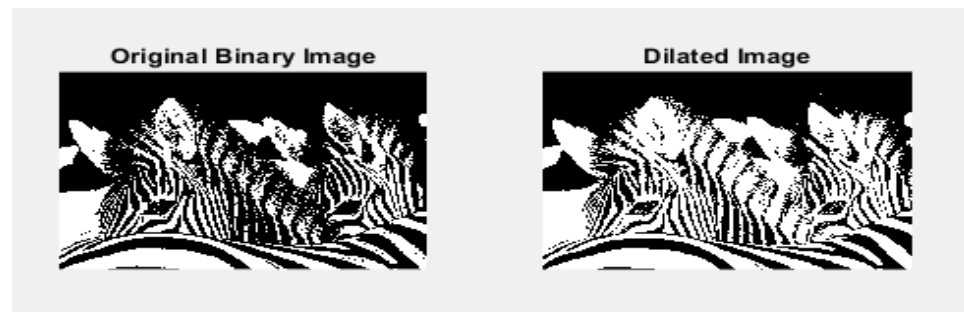
a. Objectives:

The objectives of dilation in image processing include expanding the size of objects in a binary image to enhance their visibility, filling in small holes or gaps to reduce noise, connecting nearby objects for better recognition, and improving boundary detection. Dilation is a fundamental morphological operation that aids in feature enhancement and serves as a preprocessing step for tasks like segmentation and object recognition, making it essential for various applications, including medical imaging, character recognition, and structural analysis.

Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert to binary image
bw = imbinarize(img);
% Define a structuring element
se = strel('disk', 5); % You can change the size and shape
% Perform dilation
dilated_image = imdilate(bw, se);
% Display the original and dilated images
figure;
subplot(1, 2, 1);
imshow(bw);
title('Original Binary Image');
subplot(1, 2, 2);
imshow(dilated_image);
title('Dilated Image');
```

Input and Output:



b. Objectives:

The objectives of erosion in image processing are to reduce the size of objects in a binary image, effectively removing small noise and isolated pixels, which helps in refining object boundaries and enhancing shape clarity. This operation aids in separating closely positioned objects, facilitating better feature extraction and analysis. Erosion serves as a foundational morphological operation that can be used in conjunction with dilation and other techniques, making it crucial for applications like medical imaging, industrial inspection, and character recognition, where precise object delineation is essential.

Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert to binary image
bw = imbinarize(img);
% Define a structuring element
se = strel('disk', 5); % You can change the size and shape
% Perform erosion
eroded_image = imerode(bw, se);
% Display the original and eroded images
figure;
subplot(1, 2, 1);
imshow(bw);
title('Original Binary Image');
subplot(1, 2, 2);
imshow(eroded_image);
title('Eroded Image');
```

Input and Output:



c. Objectives:

The objectives of opening and closing operations in image processing are to refine and enhance the shapes of objects within a binary image. Opening, which combines erosion followed by dilation, is used to remove small noise and separate connected objects, helping to smooth object boundaries and eliminate small artifacts. Closing, on the other hand, involves dilation followed by erosion, and is effective for filling small holes and gaps within objects, thereby enhancing their connectivity. Together, these morphological operations facilitate better image analysis, segmentation, and feature extraction in applications such as medical imaging, object recognition, and industrial inspection, where accurate representation of shapes is crucial.

Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
```

```

    img = rgb2gray(img);
end
% Convert to binary image
bw = imbinarize(img);
% Define a structuring element
se = strel('disk', 5); % You can change the size and shape
% Perform opening
opened_image = imopen(bw, se);
% Perform closing
closed_image = imclose(bw, se);
% Display the original and processed images
figure;
subplot(1, 3, 1);
imshow(bw);
title('Original Binary Image');
subplot(1, 3, 2);
imshow(opened_image);
title('Opened Image');
subplot(1, 3, 3);
imshow(closed_image);
title('Closed Image');

```

Input and Output:



d. Objectives:

The objectives of performing a Fast Fourier Transform (FFT) on an image are to analyze its frequency components, enabling the transformation of spatial information into the frequency domain. This allows for various applications such as filtering, compression, and image enhancement. By decomposing an image into its frequency components, FFT facilitates the identification of patterns, noise reduction, and the application of frequency-based filters (e.g., low-pass and high-pass filters) to manipulate specific features. Ultimately, FFT is a powerful tool for improving image quality, enabling efficient processing, and aiding in tasks like feature extraction and image reconstruction in diverse fields such as medical imaging, telecommunications, and computer vision.

Source Code:

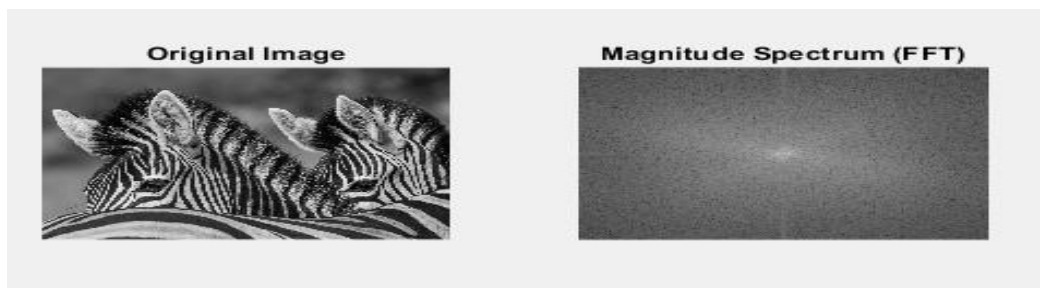
```

% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert to double for FFT processing
img = double(img);
% Compute the FFT of the image
fft_image = fft2(img);
% Shift the zero-frequency component to the center

```

```
fft_shifted = fftshift(fft_image);
% Compute the magnitude spectrum
magnitude_spectrum = log(1 + abs(fft_shifted));
% Display the original image and its FFT magnitude spectrum
figure;
subplot(1, 2, 1);
imshow(img, []);
title('Original Image');
subplot(1, 2, 2);
imshow(magnitude_spectrum, []);
title('Magnitude Spectrum (FFT)');
```

Input and Output:



e. Objectives:

The objectives of applying a homomorphic transform to an image are to enhance its contrast and improve visibility by simultaneously manipulating its illumination and reflectance components. This process transforms the image into the frequency domain, allowing for the separation of low-frequency (illumination) and high-frequency (reflectance) information. By applying appropriate filters, homomorphic transforms can reduce the effects of uneven lighting and enhance fine details, resulting in improved image quality and clarity. This technique is particularly useful in applications such as medical imaging, remote sensing, and object recognition, where consistent illumination is crucial for accurate analysis and interpretation.

Source Code:

```
% Read the input image
img = imread('C:\Users\CSELAB-2\Desktop\LAB07\Dia.jpg'); % Replace with your image file
% Convert to grayscale if it's a color image
if size(img, 3) == 3
    img = rgb2gray(img);
end
% Convert to double for processing
img = double(img) / 255; % Normalize the image
% Parameters for the homomorphic transform
gammaL = 0.3; % Low frequency gain
gammaH = 2.0; % High frequency gain
cutoff = 30; % Cutoff frequency for the filter
% Compute the FFT of the image
fft_image = fft2(img);
fft_shifted = fftshift(fft_image);
% Create a Gaussian filter
[M, N] = size(img);
[x, y] = meshgrid(-N/2:N/2-1, -M/2:M/2-1);
D = sqrt(x.^2 + y.^2);
H = 1 - exp(-(D.^2) / (2 * (cutoff^2))); % Homomorphic filter
% Apply the filter to the shifted FFT
```

```

filtered_fft = H .* fft_shifted;
% Inverse FFT to get back to the spatial domain
inverse_fft = ifft2(fftshift(filtered_fft));
enhanced_img = abs(inverse_fft);
% Apply gamma correction
enhanced_img = (gammaH * enhanced_img) ./ (gammaL * (1 - enhanced_img));
% Normalize the enhanced image
enhanced_img = mat2gray(enhanced_img);
% Display the original and enhanced images
figure;
subplot(1, 2, 1);
imshow(img, []);
title('Original Image');
subplot(1, 2, 2);
imshow(enhanced_img, []);
title('Enhanced Image (Homomorphic Transform)');

```

Input and Output:

